ANÁLISE DE PROJETO E SISTEMAS — PROJETO FINAL · DOCUMENTO DE APOIO (AID)

Hospital Connect

Plataforma Integrada de Telemedicina e Gestão Hospitalar

Da Concepção ao Dia 1 do Dev

Equipe: Igor Adriano · Rayson Jonatha · Luiz Gabriel
Data: 09/11/2026 | **Domínio:** Saúde (telemedicina e agendamento hospitalar)

SUMÁRIO

1. Concepção e Escopo ..... p. 1
2. Requisitos ..... p. 1–2
3. Arquitetura de Software (ADR, C4) ..... p. 2–3
4. Diagramas UML (6 tipos) ..... p. 4–6
5. Banco de Dados ..... p. 6–7
6. Infraestrutura e DevOps (opcional) ..... p. 7–8
7. Planejamento e Estimativas ..... p. 8
8. Documentação para o Dev ..... p. 8–9
9. Qualidade e Processo ..... p. 9

1. Concepção e Escopo — o que vamos construir e por quê

1.1 Documento de Visão

Hospital Connect é uma plataforma SaaS multitenant de gestão hospitalar integrada e telemedicina de alta disponibilidade, construída com microsserviços desacoplados por mensageria (RabbitMQ) e banco relacional PostgreSQL. Atende aos mínimos do trabalho: **4 atores** e mais de 12 funcionalidades (14 requisitos funcionais).

Problema real: longa espera no pronto-socorro virtual, histórico de saúde fragmentado (o paciente repete seu histórico a cada atendimento) e a necessidade de conformidade com a LGPD e com as normas do CFM.

Proposta de valor: agendamento autônomo, teleconsulta por WebRTC, prontuário eletrônico (PEP) centralizado, receita digital com certificado ICP-Brasil e operação com 99,9% de disponibilidade.

Fora do escopo desta versão: transcrição de áudio por Inteligência Artificial (RF14) e regra de bloqueio de cancelamento com menos de 2 horas sem justificativa (RF13) — ambos classificados como *Won't Have*.

1.2 Mapa de Stakeholders

Stakeholder	Interesse no projeto	Influência
Paciente (externo)	Agendar sem telefonar, ver prontuário e receitas no celular.	Alta
Médico / Prof. de Saúde	Atendimento remoto ágil, receita assinada sem fricção, histórico à mão.	Alta
Recepcionista	Validar cadastros, confirmar presenças e alocar salas.	Média
Administrador Hospitalar	SLA de 99,9%, LGPD, auditoria e controle de custo em nuvem.	Alta
CFM, ICP-Brasil, pagamentos	Regulação médica, validade da assinatura e cobranças.	Alta

1.3 Personas

Dr. Roberto Mendes, 42, cardiologista (telemedicina).
Motivação: agilidade, assinar receitas sem fricção, acesso rápido ao histórico. **Dor:** sistemas lentos que travam durante a chamada de vídeo.

Juliana Lima, 29, analista de marketing (paciente).
Motivação: agendar sem telefonar, receber alertas, ter receitas no celular. **Dor:** perder receitas impressas e repetir o histórico.

Carlos Eduardo, 50, diretor de operações de TI (administrador).
Motivação: 99,9% de uptime, LGPD e custo sob controle. **Dor:** instabilidade em picos e falta de trilha de auditoria.

1.4 Estudo de Viabilidade

Técnica: viável com serviços gerenciados da AWS (EKS, RDS, ElastiCache, S3) e integrações padronizadas (WebRTC, ICP-Brasil). **Operacional:** equipe de 6 pessoas e entrega em 5 sprints de 2 semanas (Bloco 7). **Custo de infraestrutura** (AWS Calculator, produção; 1 USD = R$ 5,00):

Componente	Recurso AWS	USD/mês	BRL/mês
Cluster Kubernetes	EKS (2 nodes t3.xlarge)	195,00	975,00
Banco de dados	RDS PostgreSQL Multi-AZ (db.r6g.xlarge)	420,00	2.100,00
Cache em memória	ElastiCache Redis (cache.m6g.large)	110,00	550,00
Armazenamento	S3 (exames e receitas, 500 GB)	15,00	75,00
Segurança e rede	WAF + CloudFront + ALB	120,00	600,00
Total		**860,00**	**4.300,00**

1.5 Riscos iniciais (matriz completa de 8 riscos no Bloco 7)

ID	Risco	Prob.	Impacto	Mitigação
R01	Indisponibilidade da API da ICP-Brasil	Média	Alto	Circuit Breaker e fila de retentativa no RabbitMQ
R02	Perda ou vazamento de dados sensíveis (LGPD)	Baixa	Extremo	AES-256, RBAC estrito, logs de auditoria imutáveis
R03	Instabilidade de banda em chamadas WebRTC	Média	Alto	Servidores TURN/STUN distribuídos, bitrate adaptativo
R04	Estouro do orçamento de nuvem	Média	Médio	Alertas no AWS Budgets e auto-scaling com limites
R06	Inconsistência de horários por concorrência	Média	Alto	Distributed Lock com Redis Redlock na reserva

2. Requisitos — o que o sistema faz e como se comporta (toda métrica com número)

2.1 Requisitos Funcionais com priorização MoSCoW

MoSCoW: **Must** (indispensável), **Should** (importante), **Could** (desejável), **Won't** (fora desta versão).

ID	Requisito	Prior.	Justificativa
RF01	Autenticação multifator (MFA) para médicos e administradores.	M	Protege dados de saúde; sustenta RNF03/RNF05.
RF02	Paciente agenda consulta presencial ou teleconsulta escolhendo especialidade e médico.	M	Função central do produto (US01).
RF03	Salas virtuais de teleconsulta com áudio/vídeo via WebRTC.	M	Sem ela não há telemedicina.
RF04	Médico registra o atendimento no Prontuário Eletrônico (PEP).	M	Histórico unificado é a proposta de valor (US02).
RF05	Receitas assinadas digitalmente com certificado ICP-Brasil.	M	Validade legal da receita (US03).
RF06	Notificações push / e-mail / SMS de confirmação e lembrete.	S	Reduz no-show, mas o fluxo funciona sem ela.
RF07	Paciente anexa exames (PDF/JPEG/PNG) antes da consulta.	S	Agiliza o atendimento; não bloqueia o fluxo.
RF08	Recepcionista faz triagem inicial e dá entrada no atendimento presencial.	S	Atende o fluxo presencial.
RF09	Dashboards de tempo médio de atendimento e no-show.	C	Apoio à gestão; entrega posterior.
RF10	Pagamento online de consultas particulares via PIX e cartão.	C	Receita adicional; depende de gateway externo.
RF11	Paciente solicita exportação ou exclusão dos seus dados (LGPD).	M	Obrigação legal (Lei 13.709/2018).
RF12	Farmacêutico valida o QR Code da prescrição eletrônica.	S	Dá confiança à receita digital.
RF13	Cancelamento com menos de 2 h sem justificativa fica fora desta versão.	W	Regra de negócio adiada.
RF14	Transcrição de áudio por IA fora desta versão.	W	Custo e complexidade fora do escopo.

2.2 Requisitos Não Funcionais mensuráveis

ID	Categoria	Métrica	Por que esta métrica
RNF01	Desempenho	API de leitura < 200 ms no percentil 95 (p95).	Evita travar a chamada de vídeo (dor do Dr. Roberto).
RNF02	Disponibilidade	SLA de 99,9% ao mês (máx. ~43 min de indisponibilidade).	Meta do administrador; exige redundância.
RNF03	Segurança	TLS 1.3 em trânsito e AES-256 em repouso.	Dados de saúde são sensíveis (LGPD).
RNF04	Escalabilidade	Até 5.000 requisições/s simultâneas sem degradar o p95.	Picos de agendamento.
RNF05	Conformidade	Armazenamento e logs de saúde alinhados à LGPD (Lei 13.709/2018).	Exigência legal; base do RF11.
RNF06	Usabilidade	Nota mínima 80 no teste System Usability Scale (SUS).	Adesão dos médicos (risco R05).

2.3 Matriz de Rastreabilidade

Requisito funcional	RNFs	Caso de uso	Serviço
RF02 — Agendar consulta	RNF01, RNF04	UC02	Scheduling Service
RF03 — Salas WebRTC	RNF01, RNF02	UC03 — Realizar Teleconsulta	Video Gateway
RF04 — Prontuário PEP	RNF03, RNF05	UC04 — Manter Prontuário	EHR Service
RF05 — Assinatura ICP	RNF03, RNF05	UC05 — Emitir Prescrição	Signature Integration (via EHR)

2.4 Histórias de Usuário (Given / When / Then)

US01 — Agendamento de consulta (5 story points). **Como** paciente cadastrado, **quero** escolher especialidade, médico e horário no calendário, **para** agendar de forma autônoma.
Given que estou autenticado em “Novo Agendamento”, **when** seleciono Cardiologia, Dr. Roberto e 20/11/2026 às 14:00, **then** o sistema reserva o horário por 10 minutos, gera o comprovante e envia notificação push.

US02 — Registro no prontuário PEP (8 story points). **Como** médico com login validado e CRM ativo, **quero** registrar a evolução durante a chamada, **para** manter o histórico médico unificado.
Given que estou em teleconsulta ativa com Juliana Lima, **when** preencho o Diagnóstico, informo o CID-10 e clico em “Salvar Evolução”, **then** os dados são criptografados com AES-256 e gravados com log de auditoria associado ao CRM.

US03 — Assinatura digital de receita (8 story points). **Como** médico em atendimento, **quero** assinar a prescrição com certificado ICP-Brasil, **para** que o paciente receba receita legalmente válida.
Given que finalizei a prescrição, **when** seleciono “Assinar via Certificado Digital” e informo a chave de validação, **then** o sistema consulta o autorizador ICP-Brasil, gera o PDF com carimbo de tempo e disponibiliza o QR Code de verificação.

3. Arquitetura de Software — como o sistema é estruturado; cada decisão registrada em ADR

3.1 Registros de Decisão de Arquitetura (ADR)

ADR 01 — Comunicação entre serviços (Aprovado)
Contexto: agendamento, prontuário, notificações e auditoria precisam se comunicar; chamadas puramente síncronas causariam acoplamento e falhas em cascata nos picos.
Decisão: REST/HTTP (OpenAPI) para consultas síncronas de baixa latência e mensageria assíncrona orientada a eventos (RabbitMQ) para notificações, logs e auditoria.
Alternativa descartada: comunicação estritamente síncrona via REST.
Consequências: (+) resiliência, desacoplamento, absorção de picos; (−) consistência eventual entre microsserviços.

ADR 02 — Persistência de dados (Aprovado)
Contexto: dados de saúde críticos exigem integridade relacional rígida (prontuários, agendamentos, vínculos médico–paciente).
Decisão: PostgreSQL 16 como banco principal e Redis como cache de sessões e lock temporário de horários.
Alternativa descartada: banco NoSQL de documentos (ex.: MongoDB), pois as relações Consulta→Paciente/Médico e Prontuário→Consulta pedem FKs e transações ACID.
Consequências: (+) ACID, JSONB para exames, suporte à LGPD; (−) réplicas de leitura quando o volume crescer.

3.2 Stack tecnológica

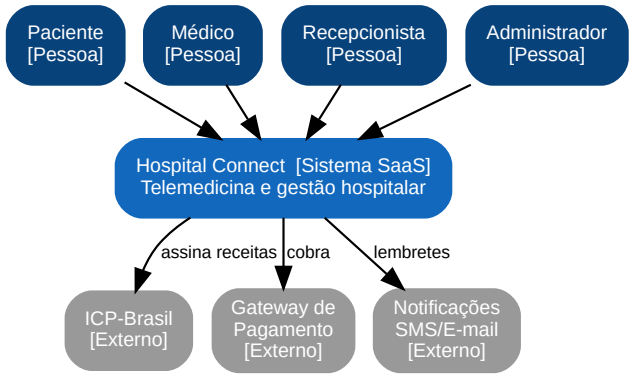
Camada	Escolha	Motivo técnico (derivado dos requisitos e ADRs)
Front-end web	**React.js (SPA)**	Interface única para pacientes e médicos.
Mobile	**Flutter**	Aplicativo nativo para iOS e Android (paciente com receitas no smartphone).
Borda / API	**Kong API Gateway**	Roteamento, autenticação, rate limiting e SSL em um único ponto (RNF03).
Serviços core	**Java 21 + Spring Boot**	Lógica de agendamento e prontuário; contrato REST/OpenAPI.
Serviços de apoio	**Node.js/TS · Go · Python/FastAPI**	Auth com MFA e JWT (Node.js/TS); notificações assíncronas de alta performance (Go, RF06); auditoria LGPD (Python/FastAPI).
Mensageria	**RabbitMQ**	Desacopla serviços e absorve picos (ADR 01).
Banco principal	**PostgreSQL 16**	ACID, integridade relacional e JSONB (ADR 02).
Cache / locks	**Redis**	Sessões e lock temporário de horários (R06).
Orquestração	**AWS EKS (Kubernetes)**	Pods dos microsserviços em 2 nodes (RNF02, RNF04).
Armazenamento	**AWS S3**	Exames e receitas (RF07).

3.3 Padrões de design aplicados

Padrão	Onde	Por quê
Event-driven (pub/sub)	RabbitMQ entre serviços	Desacoplamento e absorção de picos (ADR 01, RNF04).
API Gateway	Kong na borda	Ponto único de autenticação, rate limit e SSL.
Circuit Breaker + retry assíncrono	Integração com ICP-Brasil	Evita falha em cascata quando a API externa cai (R01, Figura 8).
Distributed Lock (Redlock)	Reserva de horário no Redis	Impede dois agendamentos no mesmo horário (R06).

3.4 Modelo C4 (4 níveis)

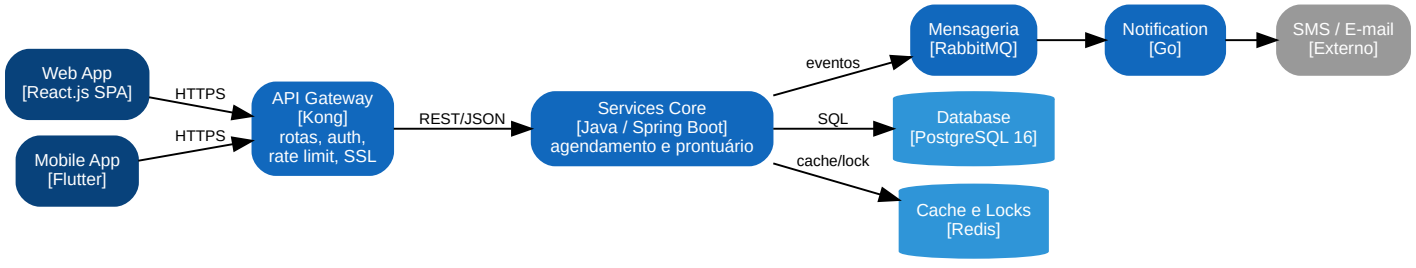
Figura 1 — C4 Nível 1 — Contexto Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026



Como ler os 4 níveis do C4

- **Nível 1 — Contexto (Fig. 1):** o Hospital Connect, seus 4 atores e os sistemas externos (ICP-Brasil, pagamento, SMS/e-mail).
- **Nível 2 — Containers (Fig. 2):** aplicações e bancos que rodam o sistema, com tecnologia e protocolo de cada ligação.
- **Nível 3 — Componentes (Fig. 5, Bloco 4):** Auth, Scheduling, EHR, Notification, Video Gateway e Audit.
- **Nível 4 — Código:** componente crítico `PrescricaoMedica.assinarDigitalmente()` (Fig. 4 e cenário de erro da Fig. 8).

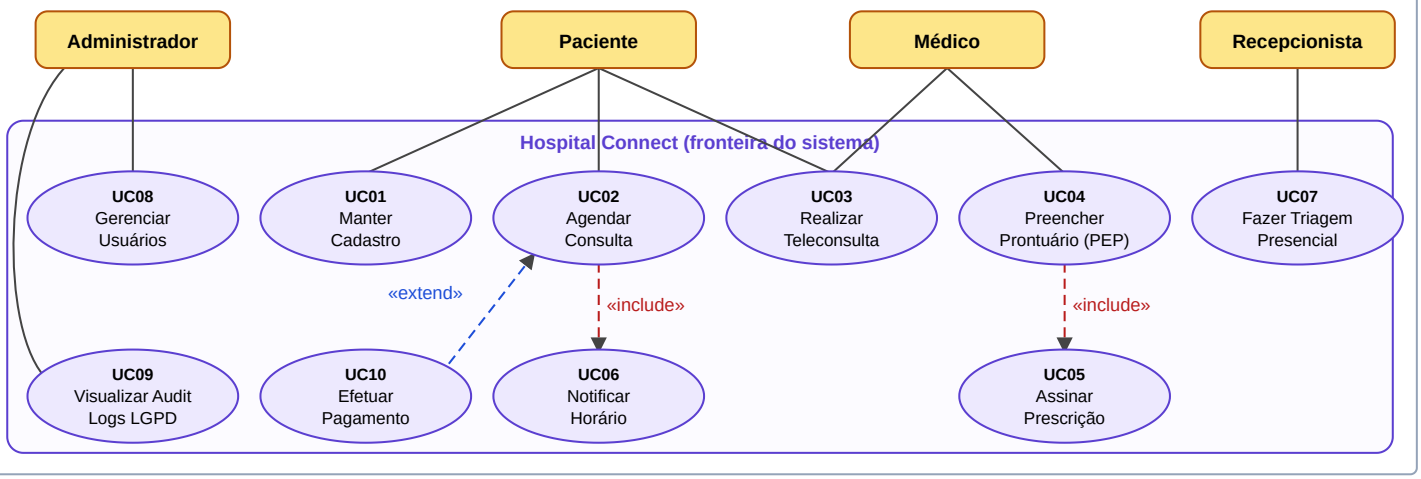
Figura 2 — C4 Nível 2 — Containers (tecnologias e protocolos) Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026



4. Diagramas UML — 6 tipos de diagrama, cada um com título, autores e data

4.1 Casos de Uso

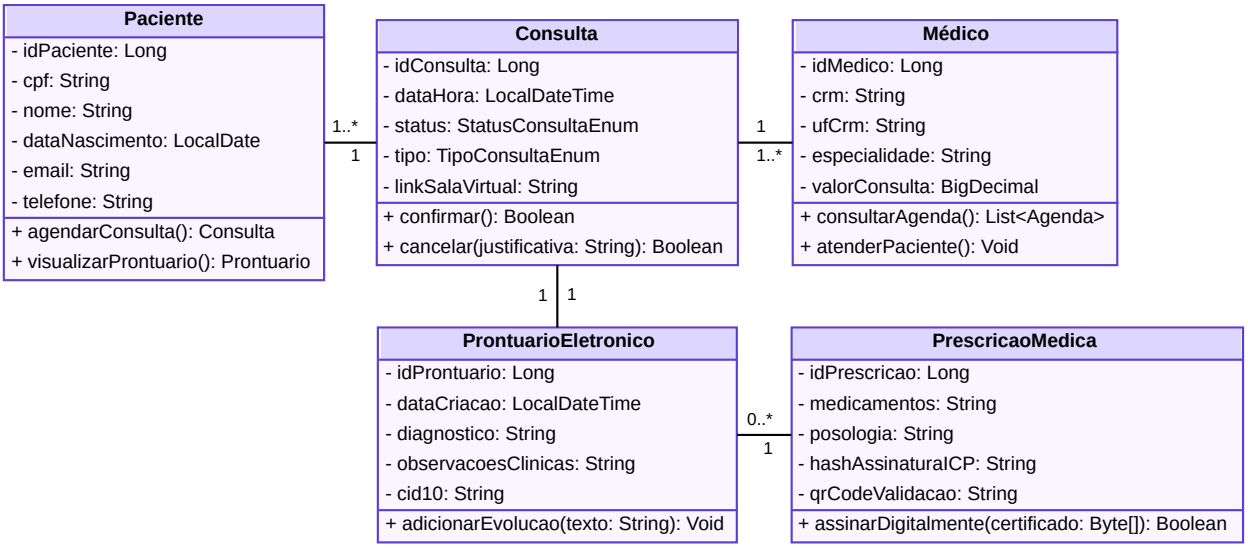
Figura 3 — Diagrama de Casos de Uso — Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026



Os quatro atores (Paciente, Médico, Recepcionista, Administrador) interagem com dez casos de uso. **UC02 inclui UC06**: todo agendamento dispara a notificação do horário (obrigatório). **UC10 estende UC02**: o pagamento só ocorre em consultas particulares (opcional). **UC04 inclui UC05**: o preenchimento do prontuário leva à assinatura da prescrição.

4.2 Classes (atributos tipados, visibilidades +/−, relações com multiplicidade)

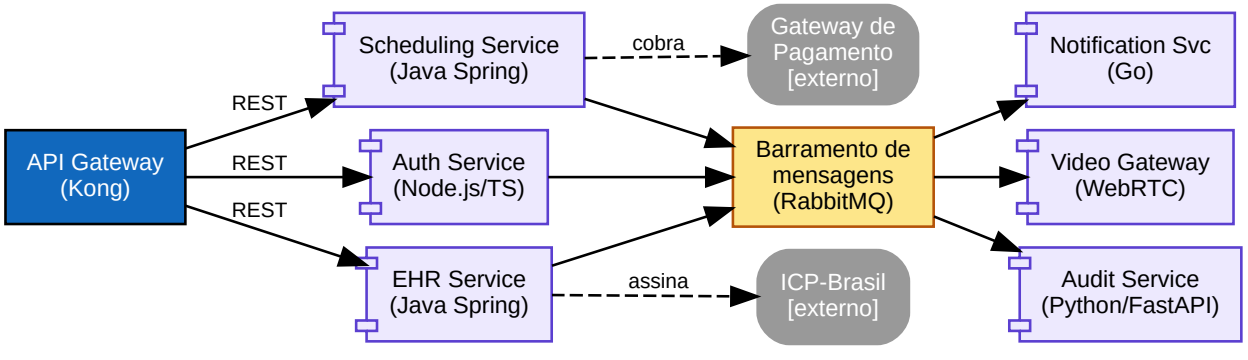
Figura 4 — Diagrama de Classes do domínio clínico — Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026



Atributos são privados (−) e **métodos públicos (+)**, preservando o encapsulamento. Um paciente e um médico têm uma ou mais consultas (1..*); cada consulta gera um prontuário (1); um prontuário pode originar zero ou mais prescrições (0..*), cada uma com `hashAssinaturaICP` e `qrCodeValidacao` para verificação.

4.3 Componentes

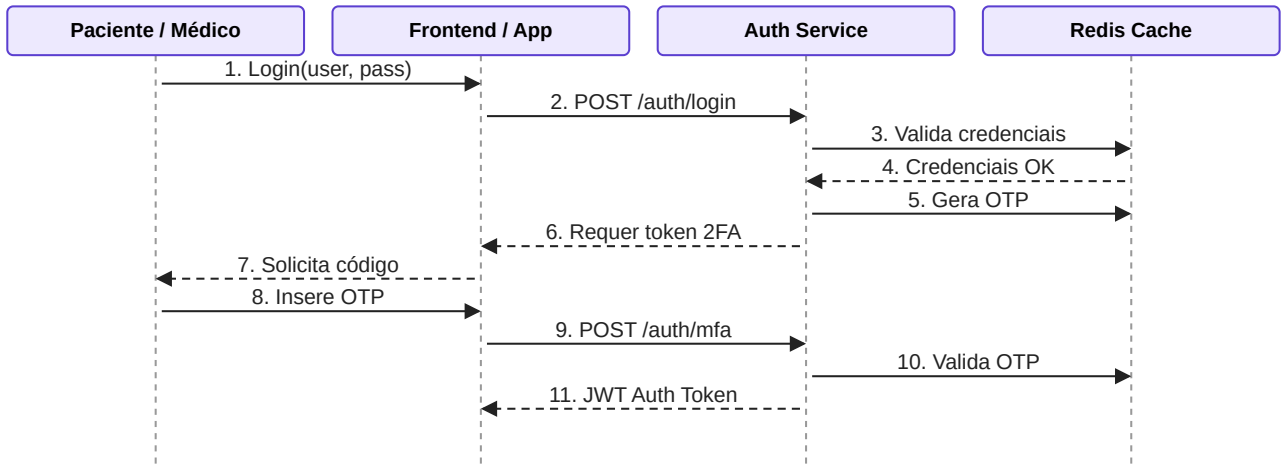
Figura 5 — Diagrama de Componentes — Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026



O Kong expõe as interfaces REST dos serviços Auth, Scheduling e EHR. Os três publicam eventos no RabbitMQ, consumidos por Notification, Video Gateway e Audit. Dependências externas: ICP-Brasil (via EHR) e gateway de pagamento (via Scheduling).

4.4 Sequência (3 diagramas obrigatórios)

Figura 6 — Sequência 1 — Autenticação com MFA Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026



Login válido não basta: o Auth Service gera um OTP (passos 3–5), o usuário o informa (8) e só então o JWT é emitido (11).

Figura 7 — Sequência 2 — Agendamento e teleconsulta (happy path) Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026

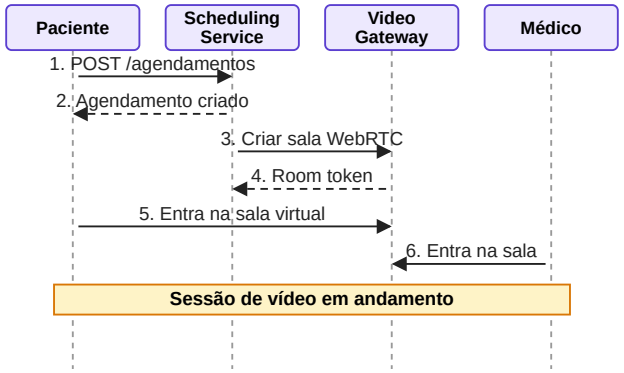


Figura 8 — Sequência 3 — Erro: falha na assinatura ICP-Brasil Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026

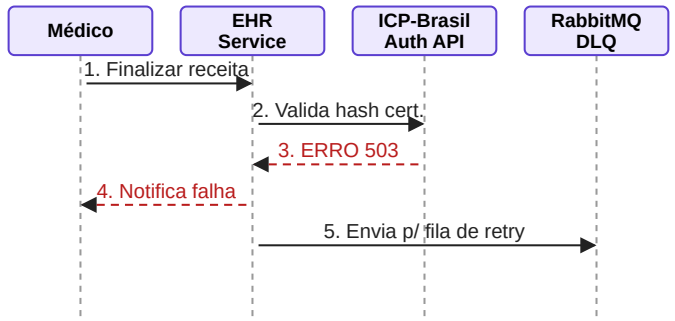
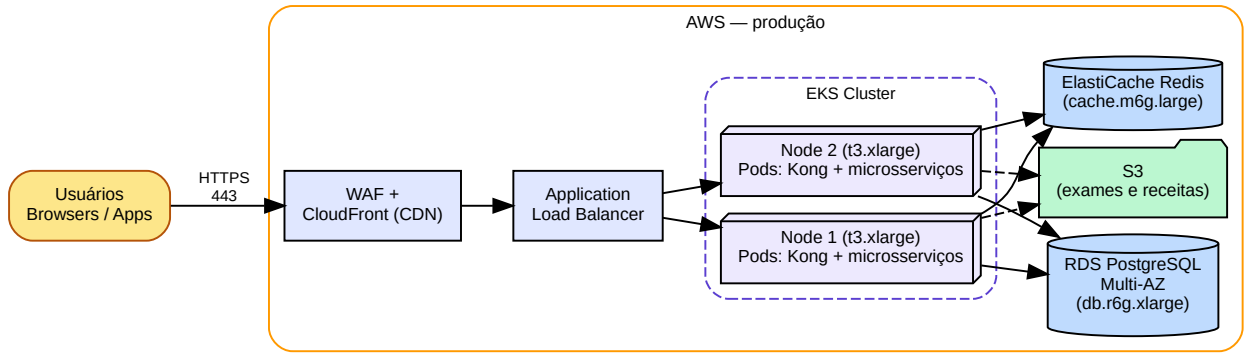


Fig. 7: a reserva cria a sala WebRTC e devolve o token a paciente e médico. **Fig. 8:** com o erro 503 da ICP-Brasil, o médico é avisado e a assinatura vai à fila de retry (DLQ) do RabbitMQ, sem perder a prescrição (R01).

4.5 Implantação (Deployment)

Figura 9 — Diagrama de Implantação — produção na AWS Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026



O tráfego HTTPS (porta 443, TLS 1.3) passa por WAF e CloudFront, chega ao ALB e é distribuído entre dois nodes EKS. Dados ficam em RDS PostgreSQL Multi-AZ e ElastiCache Redis; exames no S3. Este AID detalha o ambiente de produção; o ambiente de desenvolvimento roda localmente via Docker Compose (Bloco 8).

4.6 Atividades (2 processos)

Figura 10 — Atividades — Processo de negócio: agendamento e pagamento Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026

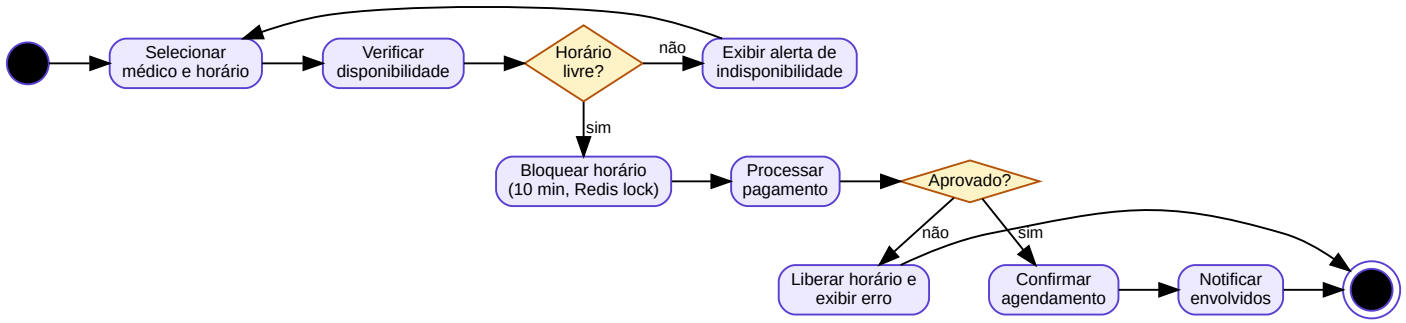


Figura 11 — Atividades — Processo administrativo: credenciamento e auditoria de médicos Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026

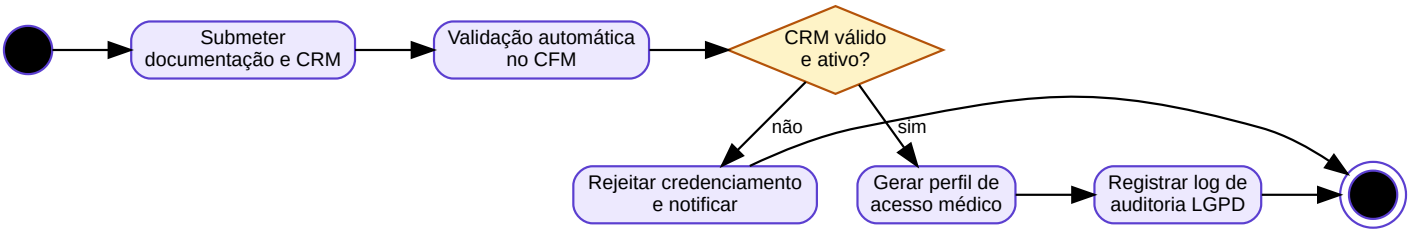


Fig. 10 (processo de negócio): o horário é bloqueado por 10 minutos durante o pagamento e liberado se recusado. Fig. 11 (processo administrativo): o CRM é validado no CFM antes de gerar o perfil e o log de auditoria LGPD.

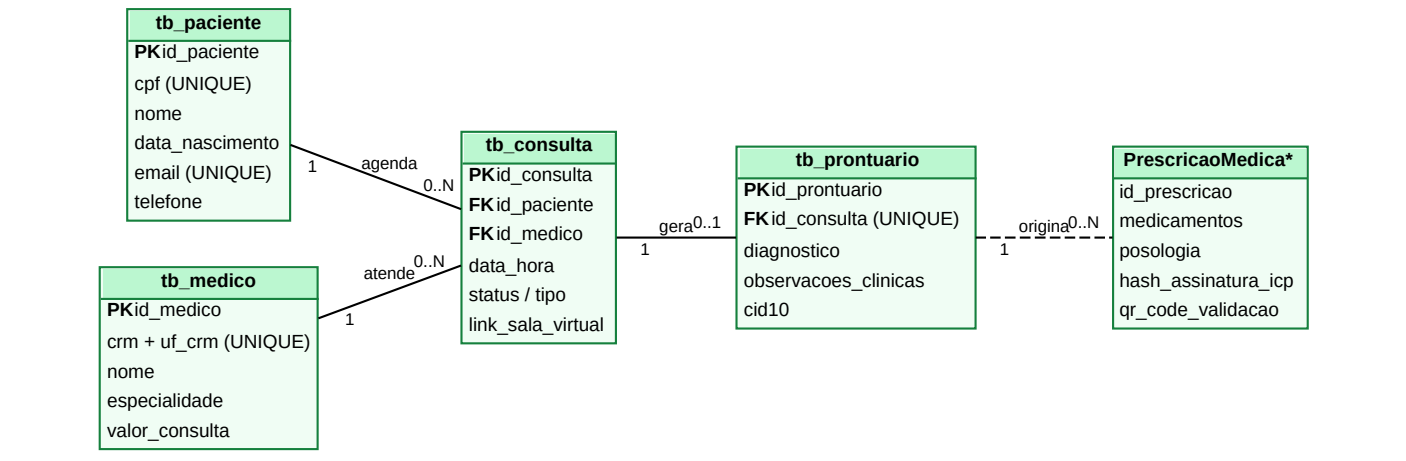
5. Banco de Dados — do modelo conceitual ao executável

5.1 Decisão do banco

Escolhido	Descartado	Motivo técnico (baseado nos requisitos)
PostgreSQL 16 (RDS Multi-AZ)	NoSQL de documentos (ex.: MongoDB)	Prontuários, agendamentos e vínculos médico–paciente exigem integridade relacional (FKs, constraints) e ACID; JSONB cobre dados dinâmicos de exames; a réplica em outra zona sustenta o RNF02 (99,9%); suporte à LGPD (ADR 02).
Redis (ElastiCache)	Lock apenas no PostgreSQL	Sessões e lock temporário de 10 minutos no agendamento (R06) com baixa latência (RNF01), sem sobrecarregar o banco transacional.

5.2 Modelo conceitual (ER)

Figura 12 — Modelo Entidade-Relacionamento (conceitual com chaves) Igor Adriano · Rayson Jonatha · Luiz Gabriel · 09/11/2026



Cardinalidades: um paciente e um médico têm de zero a N consultas; cada consulta gera no máximo um prontuário (relação 1:1 reforçada por `UNIQUE` em `id_consulta`). ***PrescricaoMedica** consta no diagrama de classes, mas ainda não possui tabela no DDL principal.

5.3 Modelo lógico e dicionário de dados (PII e dados sensíveis marcados para a LGPD)

tb_paciente

Coluna	Tipo	Restrições	LGPD
`id_paciente`	BIGSERIAL	PK	—
`cpf`	VARCHAR(11)	UNIQUE, NOT NULL	**PII** (cripto.)
`nome`	VARCHAR(150)	NOT NULL	**PII**
`data_nascimento`	DATE	NOT NULL	**PII**
`email`	VARCHAR(100)	UNIQUE, NOT NULL	**PII**
`telefone`	VARCHAR(20)	NOT NULL	**PII**
`data_criacao`	TIMESTAMPTZ	DEFAULT now	—

tb_medico

Coluna	Tipo	Restrições	LGPD
`id_medico`	BIGSERIAL	PK	—
`crm / uf_crm`	VARCHAR(20) / (2)	NOT NULL, UNIQUE(crm,uf)	—
`nome`	VARCHAR(150)	NOT NULL	**PII**
`especialidade`	VARCHAR(80)	NOT NULL	—
`valor_consulta`	NUMERIC(10,2)	NOT NULL	—

tb_consulta

Coluna	Tipo	Restrições	LGPD
`id_consulta`	BIGSERIAL	PK	—
`id_paciente`	BIGINT	FK→tb_paciente, NOT NULL	—
`id_medico`	BIGINT	FK→tb_medico, NOT NULL	—
`data_hora`	TIMESTAMPTZ	NOT NULL	—
`status / tipo`	VARCHAR(30) / (20)	NOT NULL	—
`link_sala_virtual`	VARCHAR(255)	NULL	—

tb_prontuario

Coluna	Tipo	Restrições	LGPD
`id_prontuario`	BIGSERIAL	PK	—
`id_consulta`	BIGINT	FK→tb_consulta, UNIQUE	—
`diagnostico`	TEXT	NOT NULL	**Sensível**
`observacoes_clinicas`	TEXT	NULL	**Sensível**
`cid10`	VARCHAR(10)	NULL	**Sensível**
`data_registro`	TIMESTAMPTZ	DEFAULT now	—

Normalização: as tabelas atendem à 3FN (cada atributo depende só da chave; paciente e médico não se repetem na consulta). **Índices:** as chaves PK e as restrições UNIQUE (cpf, email, crm+uf, id_consulta) criam índices automaticamente. PII = dado pessoal; Sensível = dado de saúde.

5.4 Modelo físico (DDL executável — PostgreSQL)

```
CREATE TABLE tb_paciente (
  id_paciente BIGSERIAL PRIMARY KEY,
  cpf VARCHAR(11) UNIQUE NOT NULL,
  nome VARCHAR(150) NOT NULL,
  data_nascimento DATE NOT NULL,
  email VARCHAR(100) UNIQUE NOT NULL,
  telefone VARCHAR(20) NOT NULL,
  data_criacao TIMESTAMP WITH TIME ZONE
    DEFAULT CURRENT_TIMESTAMP
);
CREATE TABLE tb_medico (
  id_medico BIGSERIAL PRIMARY KEY,
  crm VARCHAR(20) NOT NULL,
  uf_crm VARCHAR(2) NOT NULL,
  nome VARCHAR(150) NOT NULL,
  especialidade VARCHAR(80) NOT NULL,
  valor_consulta NUMERIC(10,2) NOT NULL,
  CONSTRAINT uk_medico_crm_uf
    UNIQUE (crm, uf_crm)
);
```

```
CREATE TABLE tb_consulta (
  id_consulta BIGSERIAL PRIMARY KEY,
  id_paciente BIGINT NOT NULL,
  id_medico BIGINT NOT NULL,
  data_hora TIMESTAMP WITH TIME ZONE NOT NULL,
  status VARCHAR(30) NOT NULL,
  tipo VARCHAR(20) NOT NULL,
  link_sala_virtual VARCHAR(255),
  CONSTRAINT fk_consulta_paciente
    FOREIGN KEY (id_paciente)
    REFERENCES tb_paciente(id_paciente),
  CONSTRAINT fk_consulta_medico
    FOREIGN KEY (id_medico)
    REFERENCES tb_medico(id_medico)
);
CREATE TABLE tb_prontuario (
  id_prontuario BIGSERIAL PRIMARY KEY,
  id_consulta BIGINT UNIQUE NOT NULL,
  diagnostico TEXT NOT NULL,
  observacoes_clinicas TEXT,
  cid10 VARCHAR(10),
  data_registro TIMESTAMP WITH TIME ZONE
    DEFAULT CURRENT_TIMESTAMP,
  CONSTRAINT fk_prontuario_consulta
    FOREIGN KEY (id_consulta)
    REFERENCES tb_consulta(id_consulta)
);
```

5.5 Estratégia de dados

- **LGPD:** campos PII e sensíveis marcados no dicionário; CPF criptografado; AES-256 em repouso; trilha de auditoria imutável (Audit Service); RF11 permite exportação e exclusão dos dados pelo titular.
- **Migrations:** versionadas com Flyway (`./mvnw flyway:migrate`). **Disponibilidade:** RDS Multi-AZ com failover.
- **Backup:** a política de frequência e retenção não está definida no AID; deve ser fechada na Sprint 1, junto do setup DevOps.

6. Infraestrutura e DevOps — bloco opcional · onde o sistema roda e como chega lá

Tema	O que está definido no AID
Infraestrutura	VPC não detalhada; ALB, WAF + CloudFront, EKS com 2 nodes t3.xlarge, RDS PostgreSQL Multi-AZ, ElastiCache Redis e S3 (Figura 9). **Custo mensal de produção: US$ 860,00 (R$ 4.300,00)**, calculado na AWS Calculator (Bloco 1.4). Redundância de nodes e Multi-AZ sustentam o SLA de 99,9% (RNF02); o ALB distribui a carga.
CI/CD	GitFlow com Conventional Commits; análise estática no SonarQube com cobertura mínima de 80%; integração na `develop` somente via pipeline sem falhas. O pipeline é montado na Sprint 1 (setup DevOps/CI-CD).
Containerização	Microsserviços em pods no EKS; no desenvolvimento local, Docker Compose sobe PostgreSQL, Redis e RabbitMQ (Bloco 8).
Segurança	MFA (RF01) e JWT emitido pelo Auth Service; RBAC estrito por perfil; TLS 1.3 e AES-256; WAF e rate limiting no Kong; trilha de auditoria imutável; assinatura ICP-Brasil nas receitas.

Tema	O que está definido no AID
Observabilidade	Audit Service consome eventos do RabbitMQ para registrar logs; alertas de orçamento no AWS Budgets (R04); métricas de negócio no dashboard do administrador (RF09: tempo médio de atendimento e no-show).

7. Planejamento e Estimativas — quanto custa, quanto tempo e quais os riscos

7.1 WBS (3 níveis) e cronograma em sprints de 2 semanas

Sprint	1.x Fase (nível 2)	Entregas (nível 3)
S1 · sem. 1–2	1.1 Fundação	1.1.1 Modelagem de requisitos · 1.1.2 Aprovação das ADRs · 1.1.3 Setup DevOps e CI/CD
S2 · sem. 3–4	1.2 Identidade	1.2.1 Autenticação MFA (RF01) · 1.2.2 Cadastros base (paciente, médico)
S3 · sem. 5–6	1.3 Agendamento	1.3.1 Engine de agendamento (RF02) · 1.3.2 Integração Redis Lock
S4 · sem. 7–8	1.4 Atendimento	1.4.1 Salas virtuais WebRTC (RF03) · 1.4.2 Prontuário PEP (RF04)
S5 · sem. 9–10	1.5 Conformidade	1.5.1 Assinatura ICP-Brasil (RF05) · 1.5.2 Testes de carga · 1.5.3 Auditoria LGPD

Estimativa de esforço: as histórias detalhadas somam **21 story points** (US01 = 5, US02 = 8, US03 = 8). A ordem segue a dependência técnica: identidade, agendamento, atendimento e assinatura. São 5 sprints (10 semanas); o custo da equipe é dimensionado em 3 meses.

7.2 Composição e custo da equipe (mediana salarial nacional de tecnologia — Glassdoor)

Perfil	Qtd	Salário mensal	Custo/mês	Projeto (3 meses)
Arquiteto de Software Lead	1	R$ 16.500,00	R$ 16.500,00	R$ 49.500,00
Desenvolvedor Sênior Backend	2	R$ 12.000,00	R$ 24.000,00	R$ 72.000,00
Desenvolvedor Sênior Frontend	1	R$ 10.500,00	R$ 10.500,00	R$ 31.500,00
Engenheiro DevOps / Infra	1	R$ 11.500,00	R$ 11.500,00	R$ 34.500,00
Analista de QA / Testes	1	R$ 7.500,00	R$ 7.500,00	R$ 22.500,00
Total	**6**		**R$ 70.000,00**	**R$ 210.000,00**

Por que este perfil: dois backends cobrem os microsserviços Java e os demais serviços; um frontend atende web e mobile; DevOps responde por EKS, RDS e CI/CD; QA sustenta a meta de 80% de cobertura e os testes de carga da Sprint 5; o arquiteto garante a coerência dos ADRs.

7.3 Matriz de Riscos (8 riscos: probabilidade × impacto × mitigação)

ID	Risco	Categoria	Prob.	Impacto	Mitigação
R01	Indisponibilidade na API externa da ICP-Brasil	Técnico	Média	Alto	Circuit Breaker e fila de retentativa assíncrona no RabbitMQ.
R02	Perda ou vazamento de dados sensíveis (LGPD)	Segurança	Baixa	Extremo	Criptografia AES-256 em repouso, RBAC estrito e logs de auditoria imutáveis.
R03	Instabilidade de banda nas chamadas WebRTC	Operacional	Média	Alto	Servidores TURN/STUN distribuídos com adaptação de bitrate.
R04	Ultrapassar o orçamento de nuvem	Financeiro	Média	Médio	Alertas no AWS Budgets e auto-scaling com limites rígidos.
R05	Baixa adesão dos médicos à interface do prontuário	Negócio	Média	Médio	Testes de usabilidade com a persona e fluxo com poucos cliques.
R06	Inconsistência de horários por requisições simultâneas	Técnico	Média	Alto	Distributed Locking com Redis Redlock na reserva.
R07	Atraso no envio de lembretes aos pacientes	Operacional	Baixa	Baixo	Provedores redundantes (Twilio + AWS SNS).
R08	Alterações regulatórias nas normas do CFM	Jurídico	Baixa	Alto	Acompanhamento jurídico contínuo e regras de negócio parametrizadas.

8. Documentação para o Dev — o desenvolvedor começa no Dia 1 sem fazer perguntas

8.1 README: do clone ao localhost

Pré-requisitos: Docker e Docker Compose, Java JDK 21, Node.js 20+ com Yarn e Git.

```
1. Clonar o repositório
git clone https://github.com/hospital-connect/hospital-connect-co
re.git
cd hospital-connect-core

2. Variáveis de ambiente
cp .env.example .env

3. Subir a infraestrutura local
docker-compose up -d db-postgres redis rabbitmq
```

```
4. Migrações do banco (Flyway)
./mvnw flyway:migrate

5. Iniciar a aplicação (modo dev)
./mvnw spring-boot:run

6. Documentação interativa da API
http://localhost:8080/swagger-ui.html
```

8.2 Demais itens para o Dia 1

Item	Definição
Variáveis de ambiente	Arquivo `.env.example` na raiz do repositório, copiado para `.env` (passo 2).
Contrato de API	REST documentado em OpenAPI e exposto no Swagger UI; a documentação é atualizada a cada entrega (DoD item 5).
Padrões de código	Guias de estilo da equipe; análise estática no SonarQube; Conventional Commits (ex.: `feat: adiciona assinatura ICP-Brasil`, `fix: corrige timeout de agendamento`).
Git workflow	GitFlow: `main`, `develop`, `feature/*` e `hotfix/*`; integração em `develop` via pipeline de CI/CD.
Sprint 1 (sem. 1–2)	Modelagem de requisitos, aprovação das ADRs e setup DevOps/CI-CD. Aceite: ADR 01 e ADR 02 aprovadas, pipeline operando e política de backup definida.

9. Qualidade e Processo — governança, testes e critérios de pronto

9.1 Modelo de domínio e glossário

O modelo de domínio é o diagrama de classes (Figura 4) e o ER (Figura 12): **Paciente** e **Médico** se encontram em uma **Consulta**, que gera o **Prontuário**, de onde saem **Prescrições** assinadas.

Termo	Significado
PEP / EHR	Prontuário Eletrônico do Paciente: histórico clínico unificado.
ICP-Brasil	Infraestrutura de chaves públicas que dá validade jurídica à assinatura digital da receita.
LGPD / PII	Lei Geral de Proteção de Dados (Lei 13.709/2018); PII é dado pessoal identificável. Dado de saúde é classificado como sensível.
CRM / CFM / CID-10	Registro do médico; Conselho Federal de Medicina; código internacional de doenças.
DLQ · SLA · p95 · SUS	Fila de mensagens com falha para nova tentativa; compromisso de disponibilidade; tempo abaixo do qual ficam 95% das respostas; escala de usabilidade (0–100).

9.2 Estratégia de testes e metas de cobertura

Nível	Abordagem e meta
Unitário	Cobertura mínima de 80%, verificada pelo SonarQube.
Integração	Cobertura de testes unitários e de integração igual ou superior a 80% (DoD); cenários de erro como a falha da ICP-Brasil (Figura 8).
Carga	Sprint 5: validar RNF04 (5.000 RPS) mantendo p95 < 200 ms (RNF01).
Usabilidade	Teste SUS com meta mínima de 80 (RNF06) e testes com a persona do médico (R05).

9.3 Definition of Ready (DoR)

1. História escrita em Given/When/Then.
2. Critérios de aceite definidos e revisados pelo PO.
3. Dependências técnicas e de API identificadas.
4. Estimativa em story points feita pela equipe.

9.4 Definition of Done (DoD)

1. Código seguindo os guias de estilo.
2. Testes unitários e de integração ≥ 80%.
3. Code review aprovado por 2 membros seniores.
4. Integração na `develop` via CI/CD sem falhas.
5. Documentação de API (Swagger) atualizada.

9.5 Metodologia: Scrum

Adotamos **Scrum** porque o projeto já está organizado em sprints fixas de 2 semanas, tem um PO que revisa critérios de aceite, usa story points para estimar e exige entregas incrementais com DoR e DoD. **Kanban** foi descartado porque seu fluxo contínuo não fixa os marcos por sprint (identidade, agendamento, atendimento, conformidade) dos quais o plano depende.

> **9.6 Coerência entre os blocos.** O SLA de 99,9% (RNF02) se sustenta em ALB, 2 nodes EKS e RDS Multi-AZ (Fig. 9), com custo de R$ 4.300/mês (Bloco 1.4); a mensageria da ADR 01 aparece nas Figs. 2, 5 e 8; o PostgreSQL da ADR 02 e a LGPD (RNF05, RF11) se refletem no DDL e no dicionário com PII e dados sensíveis; as 5 sprints de 2 semanas cabem na equipe de 6 pessoas (R$ 210.000 em 3 meses).